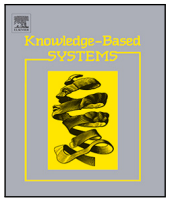
Knowledge-Based Systems 293 (2024) 111709



Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys





A spatio-temporal graph neural network for fall prediction with inertial sensors

Shu Wang [a], Xiaohu Li [b], Guorui Liao [b], Jiawei Liu [b], Changbo Liao [c], Ming Liu [d], Jun Liao [b], Li Liu [b,*]

[a] School of Materials and Energy, Southwest University, Chongqing 400715, China
[b] School of Big Data & Software Engineering, Chongqing University, Chongqing 400030, China
[c] School of Marine Science and Technology, Northwestern Polytechnical University, Xian 710072, China
[d] Faculty of Education, School of Educational Technology, Southwest University, Chongqing 400716, China

ARTICLE INFO

Keywords:
Fall prediction
Spatio-temporal dependency
Inertial sensors
Graph neural network

ABSTRACT

Falls are the leading cause of unintentional human injury, having become a public health event of strong social concern. The fall prediction technology based on wearable inertial sensors is a relatively reliable solution in human activity monitoring, a user scenario with mobility and high information privacy sensitivity, and has the advantages of low cost, small size, and high precision. However, a key challenge in sensor-based fall prediction is the fact that a fall event can often occur in various configurations of fall poses together with their own spatio-temporal dependencies. This leads us to define a spatio-temporal model to explicitly characterize these internal configurations of poses. In particular, we introduce a graph neural network with spatio-temporal topological structure to encode such latent relations among poses by capturing representative patterns in fall events. Moreover, a human body orientation estimator is devised to represent human low limbs information and as a result, separate pose dependencies are globally consistent. Empirical evaluations on two benchmark datasets and one in-house dataset suggest our approach significantly outperforms the state-of-the-art methods.

1. Introduction

According to a report from the World Health Organization, approximately 684,000 people worldwide die from accidental falls every year, and falls have become the main cause of human accidental injuries [1]. As elderly people age, their bone density and muscle mass decrease, and their functional reflexes slow down. Related research shows that approximately 27.5% of elderly people aged 65 and above in the United States face at least one fall event per year [2]. The injuries suffered by elderly people due to falls, such as physical trauma and mental illness, have a profound impact on their families, medical institutions, and the entire society [3]. Therefore, an accurate and efficient automatic fall recognition device can greatly help provide emergency assistance to elderly people who have fallen.

As shown in Fig. 1, typical fall actions are generally divided into four phases by researchers: *Pre-fall*, *Falling*, *Impact*, and *Post-fall* phases [4,5]. Fall detection and fall prediction are actually two different tasks in fall recognition. The goal of fall detection is for the system to recognize different types of fall activities, which is essentially a multi-classification task. The detection model is mostly based on the overall division and recognition of different types of falls. On the other hand, fall prediction usually identifies potential fall action before a part of the human body hits the ground, focusing on the trend of fall action during various human activities, typically within a time range of 200 to 400 ms (i.e. the time interval between an individual's imbalance and impact) [6], and then triggering reasonable protective measures to alleviate the impact of a fall on the ground, in order to obtain better injury relief.

According to different collection devices of human motion data, fall recognition schemes can be divided into recognition schemes based on environmental sensors and recognition schemes based on wearable sensors [7]. Fall recognition schemes based on environmental sensors are usually applied in non-contact and non-intrusive fall monitoring scenarios, such as cameras, WiFi, microphones, etc. Generally, users need to interact with sensing devices typically located at fixed points within their covered sensing area, which limits their perception range, portability, and privacy intrusion due to issues such as ambient light, spatial angle, and shadow occlusion. On the other hand, fall recognition schemes based on wearable sensors can ignore human mobility, and

* Corresponding author.
E-mail addresses: shuwang@swu.edu.cn (S. Wang), xhlee@cqu.edu.cn (X. Li), guoruiliao@stu.cqu.edu.cn (G. Liao), liujoy@cqu.edu.cn (J. Liu), 17317244365@189.cn (C. Liao), mingliu@swu.edu.cn (M. Liu), liaojun@cqu.edu.cn (J. Liao), dcsliuli@cqu.edu.cn (L. Liu).

https://doi.org/10.1016/j.knosys.2024.111709
Received 17 October 2023; Received in revised form 20 March 2024; Accepted 23 March 2024
Available online 25 March 2024
0950-7051/© 2024 Elsevier B.V. All rights reserved.

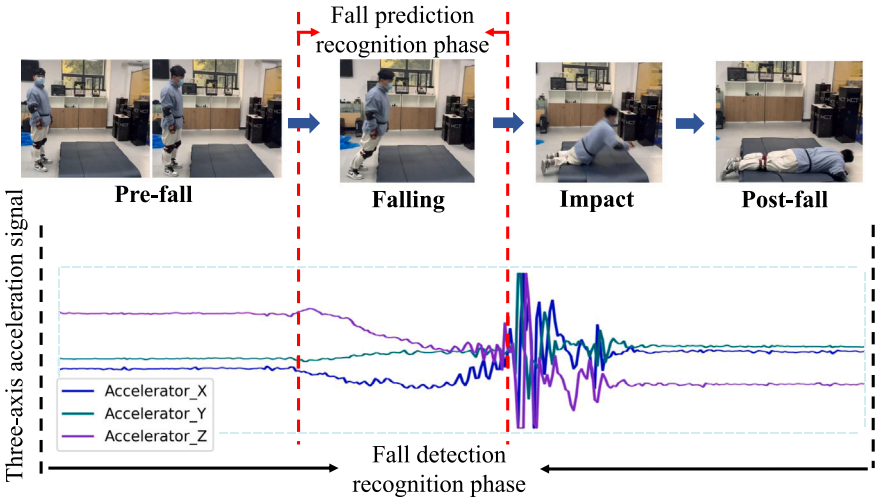


Fig. 1. Different phases of a fall process, including *Pre-fall*, *Falling*, *Impact*, and *Post-fall* phases.

the use of motion sensors can operate without any external references or restrictions, accurately and conveniently collecting human motion. The most popular motion sensors, such as Inertial Measurement Unit (IMU), currently include accelerometers, gyroscopes, and magnetometers, which can measure the acceleration and angular velocity signals of the measured object during three-dimensional movement, as well as the current geomagnetic field signals. Both computational and deployment costs have significant advantages. Wu [8] et al. proposed a novel Gated Recurrent Units (GRU) model to extract feature from mobile sensor data, reaching a prediction accuracy of 90.69%. Ashry [9] focused on recognizing different activities in a continuous stream from IMU mounted on smartwatch, using Bi-LSTM classifier.

Although fall recognition schemes utilizing motion sensors can overcome many of the shortcomings of other sensing devices, their inherent data characteristics, such as small data volume and fuzzy features, also pose great challenges for workers. Most researchers perform traditional pattern recognition based on the features of the original motion signals in the time or frequency domain, or implement recognition based on deep neural network modeling approaches. However, few consider the ST relationships contained in the motion signals related to human motion, such as the limitations and correlations of joints in human kinematics. Therefore, the following questions exist in the current field of fall recognition:

- **RQ1:** How to obtain human motion representations from simple motion signals?
- **RQ2:** How to comprehend the inter-joint topological relationships during continuous human motion?
- **RQ3:** How to obtain contextual information in complex chain movements?

To address the above questions in fall prediction, we present a graph-based recurrent neural network framework (RPTGNet) driven by falling events, as illustrated in Fig. 2. We regard the *Falling* phase as an independent fall event (making the fall action fine-grained), and use the topology structure to characterize the context dependence of the fall event and the ST relationship of the lower limb movement, trying to reduce the false or miss alarm rate of fall prediction models by finding uniform representative patterns in a variety of complex fall activities.

Our major contributions are summarized as follows:

- An limbs orientation calculation module providing real-time human lower limbs activities types with their 3D motion tracking using 3 motion sensors.
- A dynamic topology reconstruction approach to explicitly capture the inherent structural varieties of lower limbs motions with the continuous events of the representative pattern transfer behind.
- A Graph Neural Network (GNN)-based model to encode the internal configurations of limbs motion by characterizing their local and global ST dependencies in chronological order.
- A novel fall dataset ConFall12 with complex fall scenes and fine-grained action partitioning.

Compared with the state-of-the-art method, our model outperforms other existing methods in fall event prediction, which is also verified during empirical evaluations.

2. Related work

In order to provide readers with clearer understanding of our research problem, Table 1 organizes the research on mainstream approaches to fall prediction based on wearable sensors.

2.1. Sensor-based fall prediction

Threshold-based methods and machine learning algorithms are widely used for fall recognition in wearable devices [7]. Traditional recognition algorithms, such as Support Vector Machine (SVM) [10], K-NearestNeighbor (KNN) [11], Hidden Markov Model (HMM) [12], etc., have attracted the attention of researchers in recent years because of their fast response to prediction problems. They usually rely on relevant kinematics expert knowledge to establish a relationship between time and space of the human torso, and use potential variable models to represent pose sequences. Chang [12] et al. used two cameras to obtain images of 3D human skeletal trunk activity, and used hidden Markov models to evaluate the probability of subjects falling due to gait imbalance at the joints of the human skeletal trunk in 3D space. However, the poses and their ST relations in these models need to be manually coded. Due to the suddenness of fall events and the complexity of fall scenarios, it is almost impossible to complete the coding work in many actual fall scenarios.

On the other hand, the modeling method based on deep learning is currently widely used in fall recognition, which can automatically generate high-level semantic features in the latent space. Due to this advantage, deep learning methods are becoming increasingly popular in the research community. They have been used in many fields and play a role comparable to that of human experts. Generally, deep learning methods using wearable motion sensor data involve preprocessing the acquired signal, extracting features from the signal segment, and training models using these features as inputs [3]. Therefore, the research in the field of fall recognition using wearable motion sensors mainly focuses on engineering optimization features to optimize the performance of models in fall recognition. Li [13] et al. utilized a multi-IMUs feature fusion approach based on Bi-directional Long Short

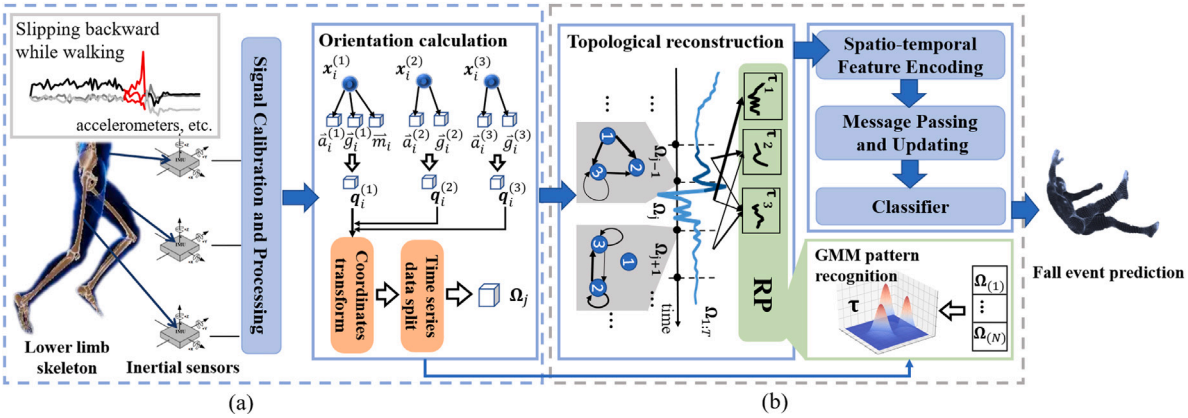


Fig. 2. RPTGNet overview. (a) An illustration of multiple inertial sensors respectively attached to lower limbs, and orientation estimator. (b) The procedure of topological reconstruction and fall event prediction.

Term Memory (Bi-LSTM) to yield an average classification accuracy of approximately 96%. Wang [4] et al. introduced a novel cascade and parallel method that employs both the characteristics of human falls and the advanced modeling ability of the Convolutional Neural Network (CNN) based self-attention. Shi [14] et al. constructed a CNN using the class activation mapping method to highlight class specific regions in the data and obtain a heatmap of the fall data, forming a special threshold method. Yu [15] et al. designed a tiny CNN for the field of fall prediction in order to strike a balance between accuracy and practicality. Chi [16] et al. proposed a CNN model based on knowledge distillation. Although a more lightweight model was obtained, the transformation process may lead to some information loss or blurring, which affects the performance of the student model. Jain [17] et al. used LSTM combined with 1D CNN to extract temporal features and introduced the concept of transition window to improve reaction time for fall prediction. Liu [18] et al. utilized the Transformer architecture feature to capture ST features of fall events. However, due to the nature of the attention mechanism, more computational resources are required. In practical recognition applications, although the temporal context information can be extracted by considering all the signals as a whole posture, the rich fine-grained spatial relations between the limbs of the human body among poses are ignored. As a result, they are not ideal for fine-grained fall prediction, especially for those with complex and diverse fall phases. Noted that the fall prediction in the main paper is different from the fall risk assessment to predict the probability of future falls, but rather an event-driven fall prediction, with the aim of quickly identifying whether there is a fall event occurring in the next sequence of phase. Events are real that occur at specific locations and times related to specific themes, often combined with fields such as disease prevention, disaster management, and business intelligence [19]. In event prediction, events within a sequence can be correlated and mutually influenced, and the prediction task is a continuous time flow, such as using traffic accident information in public transportation datasets to predict road congestion events that may occur a few minutes later [20].

2.2. Motion representation in 3D space

In recent years, many researchers have rapidly grown interest in the application of techniques such as Simultaneous Localization and Mapping (SLAM) and Robot Control to human motion representation [27]. The commonly used representation methods of object motion in 3D space include rotation matrix $\mathbf{R} = \{r_{ij}\}$, $i, j \in \{1, 2, 3\}$, Euler angle $[\psi\ \theta\ \varphi]^{\mathrm{T}}$ and unit quaternion $\mathbf{q} = [q_0\ q_1\ q_2\ q_3]$, which can be converted to each other:

$$\begin{bmatrix} \psi \\ \theta \\ \varphi \end{bmatrix} = \begin{bmatrix} \tan^{-1}\left(2\left(q_0q_1 + q_2q_3\right), 1 - 2\left(q_1^2 + q_2^2\right)\right) \\ \sin^{-1}\left(2(q_0q_2 - q_1q_3)\right) \\ \tan^{-1}\left(2\left(q_0q_3 + q_1q_2\right), 1 - 2(q_2^2 + q_3^2)\right) \end{bmatrix} \tag{1}$$

$$\mathbf{R} = \begin{bmatrix} 1 - 2(q_2^2 + q_3^2) & 2(q_1q_2 - q_0q_3) & 2(q_1q_3 + q_0q_2) \\ 2(q_1q_2 + q_0q_3) & 1 - 2(q_1^2 + q_3^2) & 2(q_2q_3 - q_0q_1) \\ 2(q_1q_3 - q_0q_2) & 2(q_2q_3 + q_0q_1) & 1 - 2(q_1^2 + q_2^2) \end{bmatrix} \tag{2}$$

$$\mathbf{q} = \begin{bmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \end{bmatrix}^{\mathrm{T}} = \begin{bmatrix} 0.5\left((r_{11} + r_{22} + r_{33})^{0.5} + 1\right) \\ \left(r_{32} - r_{23}\right)4q_0 \\ \left(r_{13} - real_{31}\right)4q_0 \\ \left(r_{21} - r_{12}\right)4q_0 \end{bmatrix}^{\mathrm{T}} \tag{3}$$

where ψ, θ and φ represent yaw angle, pitch angle and roll angle, respectively.

However, using only Euler angles has the issue of gimbal lock and non-intrinsic singularity, which has ambiguity in actual limb motion, while the rotation matrix representation takes up more space and memory, and the unit quaternion can parameterize the rotation matrix without singularity. Because of its excellent performance independent of 3-dimensional coordinate and low computational cost, the quaternion iterative algorithm can be well applied in embedded systems. Li [28] et al. used a quaternion-based fusion algorithm to monitor and track dexterous hand motion, solving the problem of line-of-sight shielding in optical measurement.

2.3. Graph structure based feature modeling

When dealing with complex time series, especially human activities sequences, the modeling method based on deep neural networks is insufficient in characterizing the spatial relationships between human limbs, neglecting the ST dependence of the human body during chain motion. Some researchers attempt to use geometric structures combined with deep learning methods to mine ST features related to the human body.

For the input modeling of GNN, the representation of human motion based on human joints and bones has a natural graph structure, containing information about different joint positions and limb rotation angles of the human body. Li [29] et al. proposed a dynamic multi-scale graph network to predict human motion based on 3D skeletons. By utilizing Graph Convolutional Network (GCN) and Temporal Convolutional Network (TCN), the ST features of skeletal points are extracted. In addition, in terms of signal sequences, it is necessary to extract interpretable features from temporal data, usually based on prior knowledge, such as physical constraints of the human body [30]. However, due to the human joint structure, it is quite difficult to track and model complex body motions by imposing such natural physical constraints. Chen [31] et al. utilized the highly interpretable Shapelet [32] to extract the internal patterns of subsequences by designing two-level temporal factors. They have constructed evolution graphs of subsequences over time, which is then merged into the time series using the graph embedding algorithm. Different from previous work, in order

Table 1
Overview of fall prediction related work. A, accelerometer; G, gyroscope; M, magnetometer; O, orientation measurement. The results of other methods applied to fall prediction will be shown in Section 5.4.

Reference	Year	Datasets used	Types of sensors	Sampling rate (Hz)	Methods	Performance
Otanasap et al. [21]	2016	self-built	A	–	Threshold	Sensitivity: 99.4% Specificity : 95.3% Accuracy : 97.4%
Pan et al. [10]	2022	self-built	A, G	–	SVM	Accuracy: 85.4%
Nandi et al. [11]	2023	Smartwatch, Notch, SmartFall [22]	A, G, M	20	KNN	Accuracy: 85.3%
Li et al. [13]	2019	self-built	A, G, M	20	Bi-LSTM	Accuracy: 90.0%
Shi et al. [14]	2020	MobiAct [23]	A, G, O	100	CNN	Accuracy: 95.3%
Yu et al. [15]	2023	SisFall [24], KFall [25]	A, G, M	200, 100	CNN	Sensitivity: 88.3% Specificity: 87.3%
Chi et al. [16]	2023	KFall [25]	A, G, M	100	CNN-ViT	Accuracy: 95.1% Precision: 80.6% Recall: 84.8% Specificity: 83.5% F1-score: 82.6%
Jain et al. [17]	2022	SisFall [24], KFallKFall [25]	A, G, M	200, 100	LSTM + 1D CNN	Accuracy: 96.3% Sensitivity: 95.5%
Liu et al. [18]	2023	SisFall [24], MobiAct [23], SoftFall [18]	A, G, M	200, 100, 25	Transformer	Accuracy: 96.7%
Ours	2024	SisFall [24], DOFDA [26], ConFall12	A, G, M	200, 33, 25	PRTG	Accuracy: 97.4% Precision: 86.5% Recall: 88.5% F1-score: 87.5%

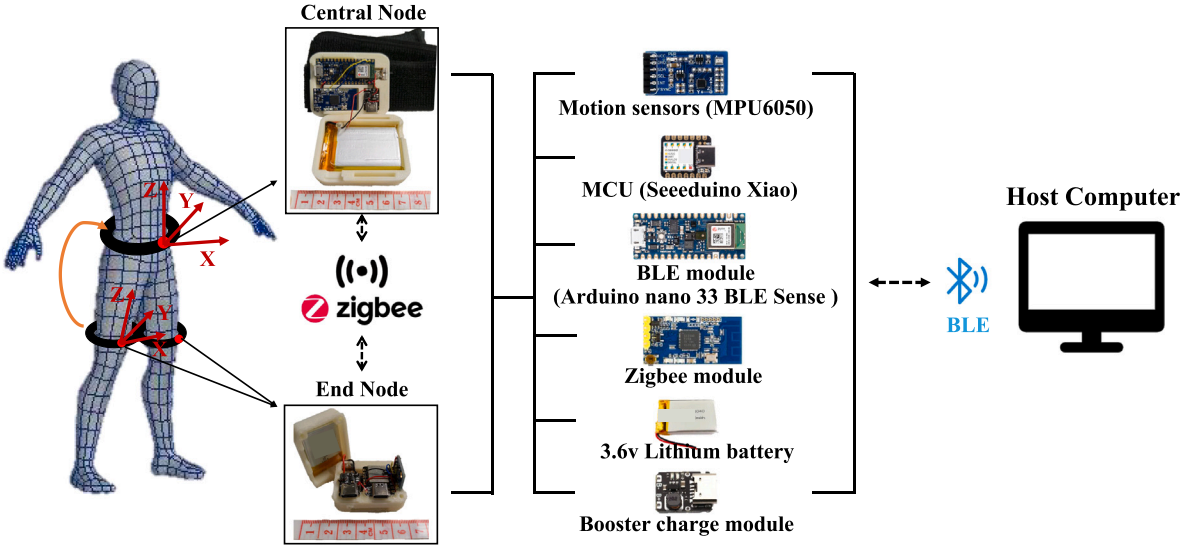


Fig. 3. Our prototype of the RPTGNet apparatus and its components.

to describe the characteristics of topological structure changes on the timeline, our proposed graph neural network is based on human limb motion information and combined with gating mechanism of LSTM to capture the transfer process of internal nodes in the directed graph and the directed edges of the overall graph structure, achieving the aggregation and transmission of topological structure messages.

3. Preliminaries

3.1. Lower limbs motion capture device design

Some researchers have compared the impact of motion signals collected from different body parts on the performance of fall recognition. For example, Pannurat [33] et al. placed accelerometers on the waist, chest, head, wrist, etc. of subjects. They believe that motion data from the waist position has the greatest impact on model performance. For the purpose of improving the quality of the fall dataset, we designed three motion capture nodes to collect motion information from the human waist and legs, respectively. The configurations of the human lower limb motion capture device is shown in Fig. 3.

Each node is composed of motion sensors, Micro-Controller Units (MCU), wireless communication modules, and power supply modules. The motion sensor uses IMU, which is connected to the MCU through I^2C bus, and the 9-axis inertial measurement unit includes a 3-axis accelerometer, a 3-axis gyroscope and a 3-axis magnetometer. To ensure the reliability and real-time performance of data transmission, we adopts the Zigbee protocol to build a tree structure network with the waist as the central node and the legs as the end nodes, so that the capture device can run independently without connecting to a host computer. In addition, the central node device placed on the waist can not only use the built-in Bluetooth Low Energy (BLE) module to interact with the host computer in real time, but also control the status of the end node devices. In order to minimize user intrusion and reduce the

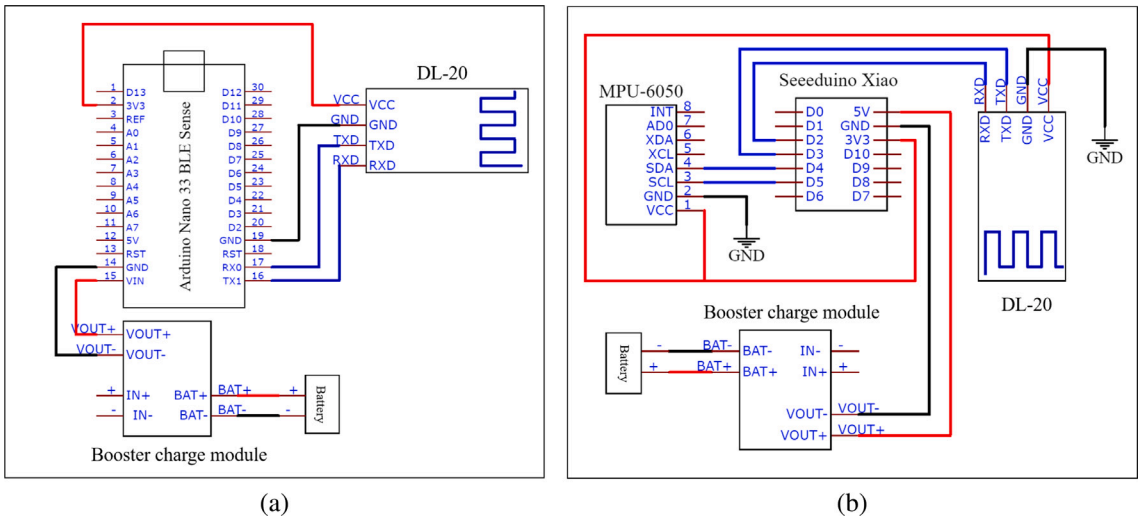


Fig. 4. Hardware circuit diagram of the lower limb motion capture device. (a) Central node data path. (b) End node data path.

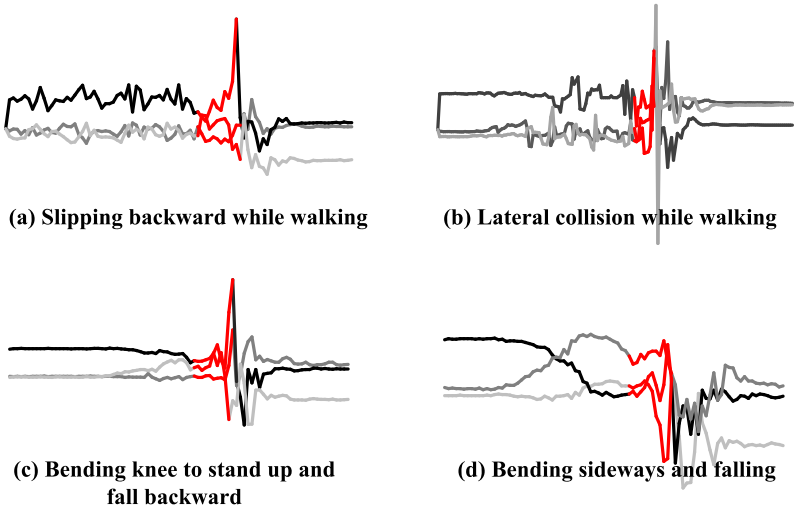


Fig. 5. Illustration of synchronous camera and motion signal: the section marked in red is *Falling* phase.

impact of the device on the subject during data acquisition, the three hardwares are designed to be compact and portable. Considering the low acquisition frequency of the magnetometer, and the introduction of the magnetometer will increase the complexity of the algorithm, we only add the magnetometer to the central node to improve the accuracy of human orientation, using the LSM9DS1 (9-axis IMU). The two end nodes are configured with an MPU6050 (6-axis IMU) connected to the micro-controller unit Seeeduino Xiao. The specific circuit connection of the lower limb motion capture device is illustrated in Fig. 4.

We designed a WIN FORM as our host computer application based on C# to read the sensor serial port and save the motion data, i.e. (3-axis acceleration + 3-axis angular velocity) × 3 motion sensors + 3-axis geomagnetic information). Due to the limitations of the wireless transmission module, the sampling frequency of the device is unified as 25 Hz.

3.2. Signal calibration and processing

Every time the lower limb motion capture device is activated, it must be properly calibrated to reduce noise during motion sensor sampling. Here, we utilized a simple calibration method [34]. The accelerometer measurement data is a mixture of gravity and motion acceleration, so when the motion sensor is stationary, first estimate the projection of gravity in the object coordinate, then separate them, and record the maximum and minimum values after separation. We convert the magnetometer data from the ontology coordinate system to the reference coordinate system, ensuring that its horizontal axis points towards the geographical north to offset the errors caused by irregular environmental magnetic fields. Finally, record the measured values of the gyroscope at rest for calibrating the gyroscope's offset error to attenuate the integration error.

Ideally, the waveform of the collected data should be a smooth curve. Due to the wireless transmission method used by the lower limb motion capture device, there may be a certain bit error rate during the transmission process, resulting in data leakage and error transmission issues. Due to the design principle of the sensor, the accelerometer is sensitive to high-frequency signals, which have a significant impact on it in the vibration environment. The noise error of the gyroscope accumulates continuously under the action of integration, ultimately leading to low-frequency interference and gyroscope drift. The magnetometer is affected by environmental magnetic fields and has poor high-frequency characteristics. Therefore, we use Kalman filters to obtain more reliable signals and avoid directional discontinuity, burrs, and other phenomena in lower limb movements, which can minimize the variance of estimation errors and iteratively update themselves as iterative filters. Compared with other filters (such as low-pass filters with fixed frequency thresholds), Kalman filters can more

accurately estimate the state affected by external interference and have low computational cost, which are usually implemented in embedded systems.

In the acquisition process of the above motion capture device, the time stamp may be missing, meaning that the sampling interval between two adjacent samples or between two different sensors may fluctuate differently. In order to stabilize the number of samples within a fixed time window, we used spline interpolation to generate samples with a fixed sampling time interval.

With the help of synchronous camera capture of images, we can accurately divide the *Falling* phase, which is the section before the fall, and fix its window size to ℓ (around 300-400 ms), indicating that the fall event has occurred. However, in other public datasets, the granularity of fall labels is coarser, and how to select the *Falling* phase is a significant challenge. After observing multiple common datasets of falls, we found that sudden signal fluctuations during the *Impact* phase of the fall process are inevitable, and the amplitude of the signal is generally large. Therefore, we adopt the Sum Magnitude Vector (SMV) algorithm [35], combined with 3-axis signal data, to calculate the amplitude of the motion signal vector and locate the most suitable sample point position. Finally, we adjust different event window sizes based on the sampling frequency of different datasets, which is illustrated in Fig. 5.

3.3. Notations and problem formulation

The lower limbs motion capture device designed in Section 3.1 includes 3 accelerometers, 3 gyroscopes and 1 magnetometer. After calibration and raw data preprocessing, the original signal at the ith sample point can be expressed as $\vec{a}_i^{\eta} = [a_{ix}^{\eta}, a_{iy}^{\eta}, a_{iz}^{\eta}]$, $\vec{g}_i^{\eta} = [g_{ix}^{\eta}, g_{iy}^{\eta}, g_{iz}^{\eta}]$, $\vec{m}_i = [m_{ix}, m_{iy}, m_i z]$, where $\eta = 1, 2, 3$ represent sensors at different positions. Over time, $\mathbf{a}^{\eta} = < \vec{a}_1^{\eta}, \vec{a}_2^{\eta}, \ldots, \vec{a}_T^{\eta} >$ is a time series collected at uniform time intervals from the η-th accelerometer, with a length of T. Similarly, $\mathbf{g}^{\eta}$ and $\mathbf{m}$ are used to represent the time series of gyroscopes and magnetometers, respectively. Therefore, the raw data at time point i is represented as $\mathbf{x}_i = [\vec{a}_i^{(1)}, \vec{g}_i^{(1)}, \vec{m}_i, \vec{a}_i^{(2)}, \vec{g}_i^{(2)}, \vec{a}_i^{(3)}, \vec{g}_i^{(3)}]$ Given a dataset $\mathbb{D} = \{\langle \mathbf{X}_{(n)}, \mathbf{Y}_{(n)} \rangle | n = 1, 2, \ldots, N\}$ composed of N time-series samples collected from 3 inertial sensors, with each sample sequence $\mathbf{X}_{(n)} = \{\mathbf{x}_i | i = 1, 2, \ldots, T\}, \mathbf{X}_{(n)} \in \mathbb{R}^{(9+6+6)\times T}$ containing T time points data, as well as the corresponding event types $\mathbf{Y}_{(n)} \in \mathbb{Z}^{T\times 1}$ $(n = 1, \ldots, N)$. It is well known that the ST relations of human limb-pose dependencies can be described by chained human motion representations. As shown in Fig. 2(a), we employ the limbs estimator to map human lower limbs from raw motion signals and transform legs' coordinates into a unified ST feature space centered on the human waist. To capture effective features from the different phases of fall activities, we divide each sequence of length T into L segments of equal size $\ell = \lfloor T/L \rfloor$:

$$< \Omega_{(n)}, \mathbf{Y}_{(n)} >= \{(\Omega_1, Y_1), \ldots, (\Omega_L, Y_L)\}, \tag{4}$$

where $\Omega_j \in \mathbb{R}^{J\times\ell}$, $j = \{1, 2, \ldots, L\}$ is a contiguous subsequence and Y_j is the corresponding observed event, called pose. Therefore, fine-grained action information is considered to be composed of motion information segments from three positions, including rotation information (three-axis attitude angle) and motion information (3-axis acceleration) of ℓ data points, with a total of 6 dimensions, i.e., $J = 3 \times 6$. The proposed neural network, based on Representative Pattern Transfer Graphs, is used to model the contextual ST dependencies between poses to predict whether the next phase is *Falling*, i.e., according to the current $(\mathscr{G}_t, Y_t) | t = 1, \ldots, L-1$, output $\hat{Y}_{t+1}$, and math it with the real Y_{t+1}. Fig. 2 illustrates our framework.

4. Our framework

In this section we provide a detailed description of how to solve points **RQ1** to **RQ3**. Table 2 indicates the notations of the formulas used throughout this paper.

Table 2
Description of the notations of the formulas used.

Notations	Description
$[\psi\ \theta\ \varphi]^{\mathrm{T}}$	Euler angle, i.e. yaw angle, pitch angle and roll angle, respectively.
$\mathbf{q}$	The unit quaternion.
$\pi(\cdot)$	An operator that converts a three-dimensional vector into an imaginary quaternion.
$\vec{\omega}_i$	The triaxial angular velocity value measured by the gyroscope.
$\vec{\omega}_\delta$	The error compensation from the accelerometer and magnetometer.
Δt	Sampling time.
$\vec{m}$	The magnetic north direction.
$\vec{a}$	The direction of gravity.
$\mathbf{C}_G^b$	Rotation matrix.
$\mathbf{q}_{am}$	Obtained from $\mathbf{C}_G^b$.
Θ	Gaussian model parameters $\langle \alpha, \mu, \Sigma \rangle$.
$\mathscr{G} = \{\mathscr{V}, E, \mathscr{W}\}$	Topological structure.
$\mathscr{V}$	Node set.
E	Edge set.
$\mathscr{W}$	The probability of RP transfer.
$\mathscr{V}$	The ST dependencies between two poses.
U	Aggregated information.
$\mathbf{W}$, $\mathbf{B}$	Weight, Bias.
M	Graph-level representation.
Y	Pose.
$\mathbf{F}_j$, $\mathbf{I}_j$, and $\mathbf{O}_j$	Forgetting gate, input gate and output gate.
σ	Sigmoid activation function.
$\mathscr{L}$	Cross-entropy loss.

4.1. Human limbs orientation calculation (RQ1)

In kinematics, the human joint is a typical hinge model, while the hip joint belongs to a spherical joint. To address **RQ1**, we designed a device with 3 motion sensors and 6 degrees of freedom to iteratively compute limb orientation in real time using unit quaternions.

The main idea of limbs orientation extraction is to use acceleration and geomagnetic signals (or individual acceleration signals) to compensate for low-frequency interference and drift caused by noise error accumulation in gyroscopes. We use the Runge–Kutta method combined with the angular velocity collected by the gyroscope to solve the differential equation of the quaternion with respect to time and realize real-time iteration. The initial value $\mathbf{q}_0$ is $[1, 0, 0, 0]$, and the update method at the sampling point $i (i \geq 1)$ is as follows:

$$\begin{aligned} \mathbf{q}_i &= \mathbf{q}_{i-1} + \dot{\mathbf{q}}\Delta t, \\ \dot{\mathbf{q}} &= \frac{1}{2}\mathbf{q}_{i-1} \otimes \pi(\vec{\omega}_i \oplus \vec{\omega}_\delta) \end{aligned} \tag{5}$$

where $\pi(\cdot)$ is an operator that converts a three-dimensional vector into an imaginary quaternion, $\vec{\omega}_i$ is the triaxial angular velocity value measured by the gyroscope. The operators $\otimes$ and $\oplus$ indicate the multiplication and combining between quaternions, respectively, and $\vec{\omega}_\delta$ is the error compensation from the accelerometer and magnetometer. When error compensation is not considered, the quaternion $\mathbf{q}_g$ can also be generated directly by using $\vec{\omega}$. Since the sampling time Δt cannot be infinitely small, the above three-axis data should be normalized to further ensure that the quaternion is a unit quaternion.

Assuming that the magnetic north direction $\vec{m}$ is the direction of the horizontal axis of the reference coordinate, and the direction of gravity $\vec{a}$ is the direction of the vertical axis of the reference coordinate, then the two are always orthogonal during any rotation of the motion sensor. From this, a rotation matrix $\mathbf{C}_G^b$ from the reference coordinate to the object coordinate is derived:

$$\mathbf{C}_G^b = [\vec{m} \quad \vec{a} \times \vec{m} \quad \vec{a}] \tag{6}$$

Unlike literature [36], which constructs the minimization problem based on the measurement values of accelerometers and magnetometers, we chose an error complementarity method with lower computational complexity. Convert $\mathbf{C}_G^b$ into quaternion $\mathbf{q}_{am}$ through Eq. (3) first, and calculate the difference $\mathbf{q}_\delta$ from the quaternion $\mathbf{q}_g$:

$$\mathbf{q}_\delta = \mathbf{q}_g^{-1} \otimes \mathbf{q}_{am} \tag{7}$$

$$\vec{\omega}_\delta = \mathrm{S}(\vec{l}_\delta) \cdot \vec{\rho}_\delta, \tag{8}$$

$$\vec{l}_\delta = 2\cos^{-1}(\mathrm{re}(\mathbf{q}_\delta)),\ \vec{\rho}_\delta = \mathrm{im}(\mathbf{q}_\delta)/\sin(\vec{l}_\delta/2)$$

Noted in the that $\mathbf{q}_g$ requires an inverse operation, rather than a rotation superposition of the two. $\mathrm{S}(*)$ is a control algorithm used to scale the error rotation axis $\vec{l}_\delta$ and angle $\vec{\rho}_\delta$, with $\mathrm{re}(*)$ and $\mathrm{im}(*)$ representing the real and imaginary parts of quaternions, respectively.

In addition, for the 6-axis motion sensor attached at the legs, we obtain a rotation matrix $\mathbf{C}_b^G$ expressed in quaternion $\mathbf{q}_g$ through the Rodrigues formula (see Eq. (2)). It can convert vectors from the object coordinate to the reference coordinate. Then, transform the gravitational acceleration $[0\ 0\ 1]^{\mathrm{T}}$ under the reference coordinate into the object coordinate through the inverse transformation of $\mathbf{C}_b^G$. There is an error between the theoretical gravity acceleration vector $\hat{a}$ in the body coordinate and the measured value $\vec{a}$:

$$\vec{e} = \vec{a} \times \hat{a}, \tag{9}$$

$$where \quad \hat{a} = (\mathbf{C}_b^G)^{-1} \times [0\ 0\ 1]^{\mathrm{T}}$$

$$\vec{\omega}_\delta = \mathrm{S}(\vec{e}) \tag{10}$$

The calculated angular velocity error $\vec{\omega}_\delta$ is used in Eq. (5).

Take the example in Fig. 2(a), the quaternion of the human legs, such as $\mathbf{q}_i^{(2)} = [q_0,\ q_1,\ q_2,\ q_3]$, is mapped to a unified ST coordinate based on the waist $\mathbf{q}_i^{(1)}$. This information can be transformed into the Euler angle:

$$\begin{aligned} \psi &= \tan^{-1}(2(ad+bc), 1-2(q_2+q_3)), \\ \theta &= sin^{-1}(2(ac-bd)), \\ \varphi &= \tan^{-1}(2(ab+cd), 1-2(q_1+q_2)) \end{aligned} \tag{11}$$

where a, b, c, d are the real numbers of the quaternion calculated from $\mathbf{q}^{(1)} \otimes \mathbf{q}^{(2)^{-1}}$. Due to the limited range of joint motion angles in the human body, in this way, the whole body can be represented in a globally consistent way rather than 3 independent coordinate axes.

4.2. Spatial–temporal dynamic topological reconstruction (RQ2)

It is known that Gaussian Mixture Model (GMM) [37] tries to find a mixed representation of the probability distribution of a multi-dimensional Gaussian model. We employ such a model to solve **RQ2** by extracting an interpretable data distribution representation from time-series data containing complex human activities, which can be formulated as:

$$p(\Omega|\Theta) = \sum_{k=1}^{K} \alpha_k \mathcal{N}(\Omega; \boldsymbol{\mu}_k, \Sigma_k) \tag{12}$$

where Θ represents all Gaussian model parameters $\langle \boldsymbol{\alpha}, \boldsymbol{\mu}, \Sigma \rangle$. $\mathcal{N}(\Omega; \boldsymbol{\mu}_k, \Sigma_k)$ represents the kth ($k = 1, \ldots, K$) component in the mixed model. α_k is the prior probability of the kth Gaussian model, satisfying $\sum_k \alpha_k = 1, \alpha_k \in [0, 1]$, representing the weight of each component. Compared with other pattern mining algorithms for time series data, GMM is suitable for multivariate time series (requiring each sampling point i.i.d), and is a probabilistic clustering method. In the process of fitting pose sequences using the GMM, the latest iteration of K Gaussian model components will be used for pattern recognition in subsequent sequences. We define these components as representative patterns (RP) $\boldsymbol{\tau}_v\,(v = 1, \ldots, K)$, $H = |\boldsymbol{\tau}_v|$. Each segment Ω_j is associated with a probability distribution $P(\boldsymbol{\tau}_v|\Omega_j)$.

Furthermore, for any two adjacent segments Ω_j and Ω_{j+1}, we introduce a topological structure $\mathscr{G}_j = \{\mathscr{V}, E_j, \mathscr{W}_j\}$ with $|\mathscr{V}| = H$ to characterize the ST dependencies between two poses, where each node represents an RP, as shown in Fig. 2(b). If there is an edge $e(v, v') \in \mathscr{E}_j$ from node v to v' with weight $w_j^{(v,v')} \in \mathscr{W}_j$, it means the probability of RP transfer from $\boldsymbol{\tau}_v$ to $\boldsymbol{\tau}_{v'}$. Inspired by the HMM, we assume that Ω_j and Ω_{j+1} are i.i.d. under the condition of using GMM parameters as hidden states. Then, $w_j^{(v,v')}$ can be understood as the joint probability distribution, as defined as follows:

$$\begin{aligned} w_j^{(v,v')} &= P(\boldsymbol{\tau}_{v,v'}|\Omega_j, \Omega_{j+1}) \\ &= P(\boldsymbol{\tau}_v|\Omega_j) \times P(\boldsymbol{\tau}_{v'}|\Omega_{j+1}). \end{aligned} \tag{13}$$

Note that the process of RP dynamic transfer with weight quantified by probability contains generally fine-grained features of various action phases (events) on both spatial and temporal dependencies.

4.3. Graph-based fall events prediction model (RQ3)

Now, we are ready to use these RP Transfer Graphs (RPTGs) and pose labels $\langle \mathscr{G}_{1:L}, Y_{1:L} \rangle$ to build a neural network to capture the contextual information about the whole fall event and make a prediction, i.e., solve **RQ3**. Rather than using simple graph-matrix vectors to obtain a high-level representation, we adopt the paradigm of the Message Passing Neural Network (MPNN) [38]. As shown in Fig. 6, two hidden representations $u_j^v \in \mathbb{R}^H$ and $M_j \in \mathbb{R}^Y$ are defined to separately encode spatial relations at the node level and temporal relations at the graph level from RP transfer graphs together with the propagation. Leveraging the existing message passing method, and combining the transfer weights between nodes, the aggregated information U_j^v from node $v \in \mathscr{V}$ at the jth RPTG can be calculated as follows:

$$U_j^v = \sum_{v' \in \mathscr{N}(v)} (\mathbf{W}^{(\mathbf{A})}[w_j^{(v,v')} \times u_{j-1}^v] + \mathbf{B}^{(\mathbf{A})}) \tag{14}$$

where $\mathbf{W}^{(\mathbf{A})}$ and $\mathbf{B}^{(\mathbf{A})}$ are the weight and bias, respectively, in the transmission of message $w_j^{(v,v')} \times u_{j-1}^v$. $\mathscr{N}(v)$ means the set of neighbor nodes of v. Initially, $u_0^v = \boldsymbol{\tau}_v$. It is not enough to obtain a spatial node level representation of the temporal changes in RPTG during falls. Referring to recurrent neural networks, the ability of GNN to process sequences can be enhanced by adding gating devices. To incorporate these global ST dependencies in a uniform way, the update of M_j and u_j^v at the current time point j needs not only the aggregated information U_j^v, but also M_{j-1} and u_{j-1}^v at the previous time point (Fig. 6):

$$\begin{aligned} u_j^v, M_j &= \mathscr{F}_{recur}(U_j^v, u_{j-1}^v, M_{j-1}) \\ &= \begin{cases} u_j^v = \boldsymbol{\Phi}_u(u_{j-1}^v, U_j^v \oplus M_{j-1}) \\ M_j = \boldsymbol{\Phi}_M(M_{j-1}, Y_j \oplus \sum_{v \in \mathscr{V}} u_j^v) \end{cases} \end{aligned} \tag{15}$$

where $\oplus$ is the concatenation operator. Notably, the current graph-level representation M_j contains the information from the past M_{j-1}, the current pose Y_j and all the inertial sensor information u_j^v. $\boldsymbol{\Phi}_*(\cdot,\cdot)$ denotes the recurrent function implemented by the LSTM gating:

$$\boldsymbol{\Phi}_u(u_{j-1}^v, U_j^v \oplus M_{j-1}) = \begin{cases} \boldsymbol{\imath}_j = U_j^v \oplus M_{(t-1)} \\ \mathbf{F}_j = \sigma(\mathbf{W}^{(\mathbf{F})} \cdot [u_{j-1}^v, \boldsymbol{\imath}_j] + \mathbf{B}^{(\mathbf{F})}) \\ \mathbf{I}_j = \sigma(\mathbf{W}^{(\mathbf{I})} \cdot [u_{j-1}^v, \boldsymbol{\imath}_j] + \mathbf{B}^{(\mathbf{I})}) \\ \mathbf{C}_j = \mathbf{F}_j \circ \mathbf{C}_{j-1} + \mathbf{I}_j \circ \tanh(\mathbf{W}^{(\mathbf{C})} \cdot [u_{j-1}^v, \boldsymbol{\imath}_j] + \mathbf{B}^{(\mathbf{C})}) \\ \mathbf{O}_j = \sigma(\mathbf{W}^{(\mathbf{O})} \cdot [u_{j-1}^v, \boldsymbol{\imath}_j] + \mathbf{B}^{(\mathbf{O})}), \quad u_j^v = \mathbf{O}_j \circ \tanh(\mathbf{C}_j), \end{cases} \tag{16}$$

where $\mathbf{F}_j$, $\mathbf{I}_j$, and $\mathbf{O}_j$ are the forgetting gate, input gate and output gate, respectively, and σ is the sigmoid activation function.

For simplicity, we have built an LSTM with a fully-connected layer for binary classification, involving end-to-end learning to predict whether the event type Y_{j+1} is the *Falling* phase.

$$u_j^G = \mathscr{F}_{fc}(M_j \oplus \sum_{v \in \mathscr{V}} u_j^v). \tag{17}$$

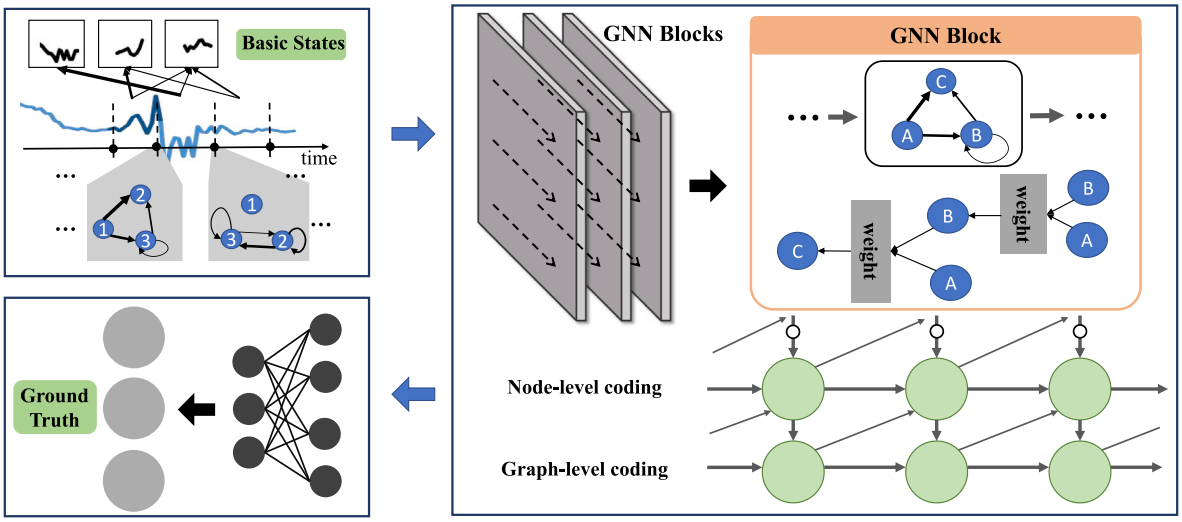


Fig. 6. Characteristics stream coding of GNN-based backbone.

Finally, the pose in the next period can be predicted by the probability $P(\hat{Y}_{j+1}|u_j^G)$. To train our model, we choose cross-entropy as the loss function:

$$\mathscr{L}(Y_{j+1}, \hat{Y}_{j+1}) = -\sum \hat{Y}_{j+1} \log P(Y_{j+1}|u_j^G) + (1 - \hat{Y}_{j+1}) \log (1 - P(Y_{j+1}|u_j^G)),$$

where $\mathbf{W} = [\mathbf{W}^{(\mathbf{A})}, \mathbf{W}^{(\mathbf{F})}, \mathbf{W}^{(\mathbf{I})}, \mathbf{W}^{(\mathbf{O})}, \mathbf{W}^{(\mathbf{C})}, \mathbf{B}^{(\mathbf{A})}, \mathbf{B}^{(\mathbf{F})}, \mathbf{B}^{(\mathbf{I})}, \mathbf{B}^{(\mathbf{O})}, \mathbf{B}^{(\mathbf{C})}]$ can be estimated by optimizing the following objective over the dataset $\mathbb{D}$:

$$\hat{\mathbf{W}} = \arg\min_{\mathbf{W},\mathbb{D}} \mathscr{L}(Y, \hat{Y}). \tag{18}$$

Due to the construction of RPTG with $|\tau|$ nodes based on the number of time series segments, the time complexity of each iteration is $O(LK^2)$, which is acceptable for fall prediction in practice.

5. Framework evaluation

5.1. Datasets

Three fall datasets were utilized in our experiments, comprising two publicly-available benchmark datasets and one in-house dataset on diverse fall scenarios collected by ourselves.

SisFALL(200 Hz) [24]. This public dataset contains a total of 4505 samples, including 15 fall activities and 19 activities of daily life (ADLs), which was collected using two accelerometers and one gyroscope placed at the waist of 38 participants. Due to the large proportion of ADLs samples, we only selected some ADLs related to fall activities. For example, we have "fall forward while walking caused by a slip", then ADL "walking slowly" will be chosen.

DOFDA(33 Hz) [26]. It is the latest in various fall datasets. By placing an accelerometer, a gyroscope, and a magnetometer on the waist of 8 participants, it collected motion data for 13 fall activities and 5 ADL, resulting in a total of 432 samples.

ConFall12(25 Hz). We propose a new dataset, ConFall12, collected by the motion capture device in Section 3.1 (see Table 3). It contains 15 fall activities and 6 ADLs. As shown in Fig. 7, we guid 12 subjects (8 males, 4 females) to perform each activity repeatedly 3 times, and collect 2592 samples. In order to test the performance of our approach to predict fall event in a complex set of activities, we categorized fall events (slipping, tripping, passing out, etc.), fall direction (vertical, front-to-back, sideways, etc.), fall state (walking, running, bending, knee bending, etc.) to construct a fall dataset containing complex scenes, such as "Slipping backward while walking", "Bending knee to stand up and fall backward". These fall activities are more in line with what actually happens in people's lives, that is, they have a variety of contextual information, and the time of fall events is short and its proportion is small. Also, with the help of a depth camera, we are able to perform fine-grained segmentations of the internal phases of fall activities.

Table 3
Description of ConFall12 dataset.

Fall/ADL	Number	Activity
ADL	1	Walking slowly.
	2	Running slowly.
	3	Picking up/bending down.
	4	Going up and down stairs.
	5	Lying-Sitting-Standing.
	6	Standing-Sitting-Lying.
FALL	1	Basic falls (four directions).
	2	Crouch-up lateral fall.
	3	Crouch-up backward fall.
	4	Bending knee to stand up and fall forward.
	5	Bending knee to stand up and fall backward.
	6	Bending forward to fall.
	7	Bending sideways and falling.
	8	Bending up and falling backward.
	9	Lying down and roll over to fall vertically.
	10	Slipping backward while walking.
	11	Tripping in forward direction while walking.
	12	Lateral collision while walking.
	13	Fainting directly to the side while walking.
	14	Tripping in forward direction while running.
	15	Slipping sideways while running.

5.2. Experimental set-ups and baselines

Our model is implemented by Tensorflow and optimized by Adam optimizer($\beta_1 = 0.7$ and $\beta_2 = 0.999$) with the learning rate of 1×10^{-3}. The batch size is fixed at 100 and the number of training epochs is 200. We set the hyperparameters $L = 10, K = 15, H = L \times K$ and $Y = K \times H$.

The performance of our approach is compared against 3 conventional pattern recognition methods: a threshold-based model [21], SVM [10], KNN [11], 4 network-based methods: Bi-LSTM [13], CHARM-deep [9], GRU [8], CMFALL [4] and 2 GNN-based methods: GCN-LSTM [39], Time2Graph [31]. For fair comparison, we have implemented a unified event prediction interface for the above models to achieve fine-grained fall prediction. No data augmentation or pre-trained weights were used to improve performance. The evaluation metrics are *accuracy*, *precision*, *recall*, *F1 score*, and *AUC* to measure the false or miss alarm rate of fall prediction.



Fig. 7. 15 fall activities in ConFall12 dataset.

5.3. Effectiveness of RPTG

As shown in Figs. 8 and 9, we conducted an observational study on the topological modeling method of motion information to explore the feasibility of this topological structure in predicting fall events, so as to help understand the explicit characteristics of human motion data at the topological level during human falling.

Unlike other fall recognition datasets, the fall dataset designed in this article is not limited to whether the participants choose to continue their activities before the fall, struggle to roll over after the fall, or remain stationary, ensuring the continuity of the data, in order to simulate the suddenness of a fall event and the most likely response of the person after the fall. The generation of representative patterns comes from all motion datasets with $K = 10$. Above Fig. 8 are two sequences of human motion postures ("Slipping backward while walking" and "Tripping in forward direction while running"), which contain information on the acceleration and orientation of the limbs. After pattern recognition, they are reconstructed into several ST dynamic graphs. For ease of differentiation, we have highlighted the sequence segments representing the *Falling* phase in red and used thicker lines to draw the directed edges with higher weights in the RPTG. At the bottom part of Fig. 8, it shows the transfer process of RP in the RPTG representing slipping ($V \rightarrow VI$, $VI \rightarrow VII$), and tripping ($VI \rightarrow VII$, $VII \rightarrow VIII$) after the motion sequence has been reconstructed by pattern recognition during the occurrence of a fall. There is a significant transfer in the RPTG among representative patterns (#0, #6, #7, #8, #9), with larger weights, and each node also has a trend towards the representative pattern (#9). Similarly, during the period represented by the RPTG ($VII \rightarrow VIII$ and $VIII \rightarrow IX$), there is no fall event that belongs to the *Post-fall* phase, and their representative patterns mainly transfer to nodes (#4 and #5).

Then, we calculated the centrality measurements of RPTG before and after the fall event: betweenness and closeness for statistical analysis. From Fig. 9, it can be clearly seen that when a fall occurs, the box chart (red bar) representing the fall is more dense, showing a lower betweenness score and a higher closeness score, while daily activities (blue bar) are very stable. These all indicate that RPTG has important information for predicting falls.

5.4. Comparison results

It is noted that our comparative experiment is carried out on hyperparameter $K = 10, L = 10$. Table 4 reports the comparison results. On the whole, our model clearly outperforms the other models with a large margin on all three datasets, about $5\% - 30\%$ performance improvement, especially in *recall* and *F1 score* indicating a smaller false or miss alarm rate. This is mainly due to its abilities to utilize spatial relations between limbs and explore rich ST dependency information between fine-grained poses. Unfortunately, threshold-based approaches lose their flexibility when confronted with complex fall scenarios. Machine learning based methods are weaker than deep learning methods in either solution space or generalization performance. And other competitive deep learning models are also quite limited in describing ST features and do not have the ability to characterize the structural relationships between human joints. It can be seen that although other models have high *accuracy*, they perform poorly in indicators such as *precision* and *recall*, which focus on positive samples.

Individually, the performance of most models on SisFALL is better than that of SisFALL, which may be due to the higher collection frequency and more categories of human activities of SisFALL. It is worth noting that ConFall12 contains a large number of falls with more challenging poses, and has more detailed pose annotations. Therefore, our model has a relatively good performance on the self-collected dataset, and the performance of the RNN-based models is more improved than that the GNN-based models. In addition, the selected GNN-based models have the ability to describe the spatio-temporal characteristics, so the accuracy performance is better than our model, but our model still has a higher *F1 score*.

5.5. Ablation study

Fig. 10(a) shows the results by varying the number of RP K and the size of prediction window L. Obviously, the model performs well when K ranges in $[3, 15]$. A higher K value implies more computational consumption, but it also results in a reduction in the edge set $|\mathscr{E}|$ without significant performance improvement. In addition, a small prediction window size can reduce the number of RPTG and cannot

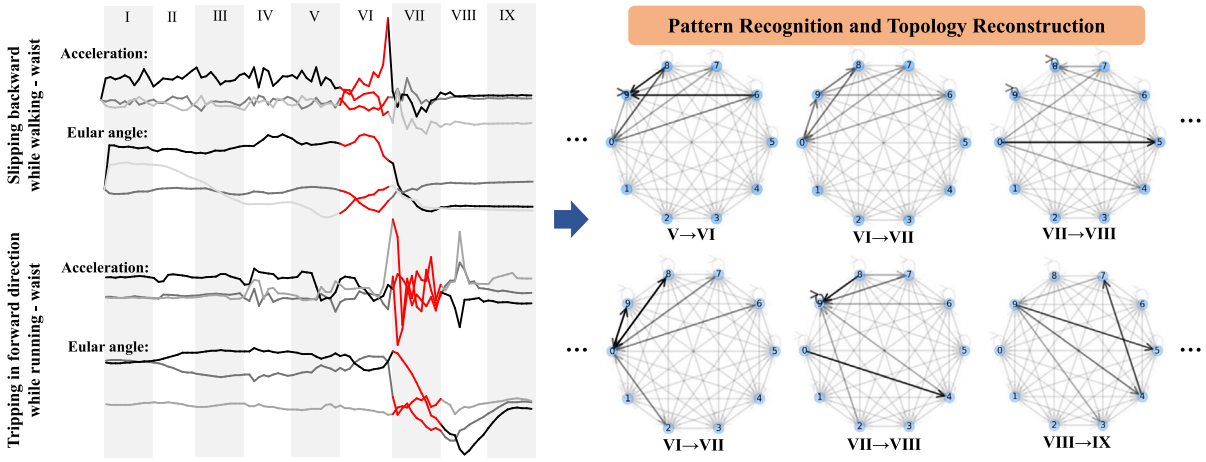


Fig. 8. Results of pattern recognition and temporal data topology: the section marked in red is *Falling* phase.

Table 4
Comparison results (%) on SisFALL, DOFDA and ConFall12.

Methods \ Datasets	SisFALL				DOFDA				ConFall12			
	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score	Accuracy	Precision	Recall	F1 score
Threshold [21]	90.7(−5.4)	39.7(−43.8)	50.8(−25.4)	44.6(−35.1)	90.6(−6.1)	34.0(−42.3)	45.3(−29.1)	38.8(−36.5)	92(−5.4)	48.5(−38.0)	54.2(−34.3)	51.2(−36.3)
SVM [10]	91.8(−4.3)	47.3(−36.2)	59.3(−16.9)	52.6(−27.1)	93.3(−3.4)	56.4(−19.9)	46.3(−28.1)	50.8(−24.5)	91.9(−5.5)	48.1(−38.4)	64.4(−24.1)	59.6(−27.9)
KNN [11]	95.5(−0.6)	69.5(−14.0)	60.3(−15.9)	64.6(−15.1)	91.5(−5.2)	45.2(−31.1)	62.7(−11.7)	52.5(−22.8)	93.6(−3.8)	56.7(−29.8)	62.7(−25.8)	58.5(−29)
Bi-LSTM [13]	95.9(−0.2)	85.6(+2.1)	70.6(−5.6)	77.4(−2.3)	96(−0.7)	72.5(−3.8)	65.3(−9.1)	68.7(−6.6)	96.1(−1.3)	81(−5.5)	80.7(−7.8)	80.8(−6.7)
CHARM-deep [9]	95.1(−1.0)	78.6(−4.9)	68.9(−7.3)	73.4(−6.3)	95(−1.7)	60.3(−16)	61.9(−12.5)	61.1(−14.2)	95.4(−2.0)	84.9(−1.6)	68.1(−20.4)	75.6(−11.9)
GRU [8]	94.3(−1.8)	74.3(−9.2)	63.2(−13)	68.2(−11.5)	95.9(−0.8)	74.4(−1.9)	57.3(−17.1)	64.7(−10.6)	95.1(−2.3)	77(−9.5)	74.4(−14.1)	75.7(−11.8)
CMFALL [4]	92.7(−3.4)	64.6(−18.9)	56.1(−20.1)	60.1(−19.6)	94.9(−1.8)	63(−13.3)	51.3(−23.1)	56.6(−18.7)	95.5(−1.9)	80.9(−5.6)	73.9(−14.6)	77.2(−10.3)
TinyCNN [15]	94.3(−1.8)	72.4(−11.1)	67.6(−5.8)	75.6(−4.1)	94.5(−2.2)	65.2(−11.1)	68.9(−5.5)	63.2(−12.1)	93.5(−3.9)	77.8(−8.7)	69.3(−19.2)	74.4(−13.1)
LSTM-1D CNN [17]	95.7(−0.4)	84.3(+0.8)	74.7(−1.5)	77.9(−1.8)	95.7(−1.0)	71.5(−4.8)	66.8(−7.6)	67.5(−7.8)	96.3(−1.1)	85.2(−1.3)	72.1(−16.4)	80.7(−6.8)
Transformer [18]	95.9(−0.2)	82.4(−1.1)	73.4(−2.8)	76.8(−0.9)	96.4(−0.3)	73.4(−2.9)	66.2(−8.2)	64.5(−10.8)	96.7(−0.7)	86.1(−0.4)	74.3(−14.2)	81.2(−6.3)
GCN-LSTM [39]	96(−0.1)	85(+1.5)	72.2(−4)	78.1(−1.6)	95.9(−0.8)	69.5(−6.8)	67.8(−6.6)	68.6(−6.7)	96.9(−0.5)	**91.7**(+5.2)	76.5(−12)	83.4(−4.1)
Time2Graph [31]	95.7(−0.4)	**85.7**(+2.2)	68.9(−7.3)	76.4(−3.3)	96.6(−0.1)	**81.1**(+4.8)	63.6(−10.8)	71.3(−4.0)	96.9(−0.5)	88(+1.5)	80.7(−7.8)	84.2(−3.3)
Ours	**96.1**	83.5	**76.2**	**79.7**	**96.7**	76.3	**74.4**	**75.3**	**97.4**	86.5	**88.5**	**87.5**

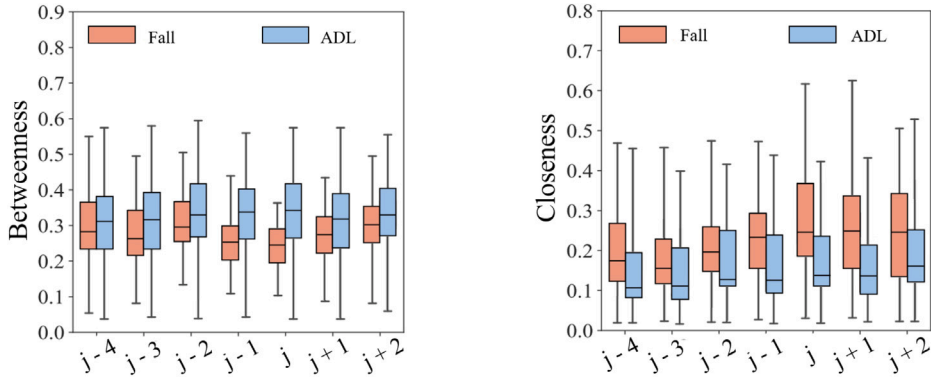


Fig. 9. The statistical analysis of RPTG.

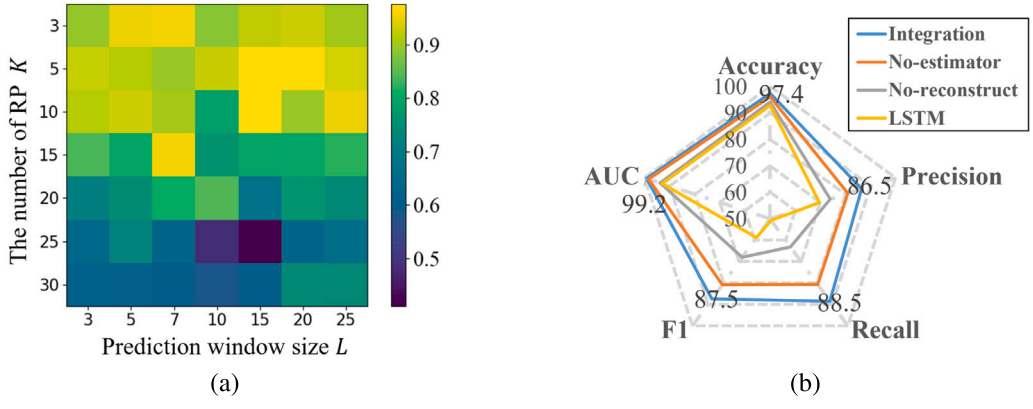


Fig. 10. Ablation study results. (a) Accuracy(%) comparison of different settings of K, L. (b) Effects of different modules on model performance.

effectively represent the ST characteristics among poses, resulting in a high miss alarm rate. Moreover, we separately evaluate the effects of different components in our network by removing modules. Fig. 10(b) investigates the components' impact on ConFall12. We can see that changing the components may lead to negative effects on the performance of our model. When removing ST topological structure, the performance drops faster than that removing orientation estimator. Besides, when replacing our encoder with LSTM, the model gives worse performance due to the limitation of managing ST dependencies.

6. Conclusion

In this paper, we present a GNN-based fall prediction method, which can capture both temporal and spatial relations among poses, especially for the *Falling* phase in complex scenes. It is more efficient than existing methods in fall prediction, particularly in reducing the miss or false alarm rates for fall events. As for future work, we will consider learning causal relations among poses for robustness and exploring the applications of our model on other phases in fall events.

CRediT authorship contribution statement

Shu Wang: Writing – review & editing, Writing – original draft, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xiaohu Li:** Software, Methodology, Data curation. **Guorui Liao:** Writing – review & editing, Software, Formal analysis, Data curation. **Jiawei Liu:** Software, Investigation, Data curation. **Changbo Liao:** Data curation. **Ming Liu:** Data curation. **Jun Liao:** Supervision, Formal analysis. **Li Liu:** Writing – review & editing, Validation, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgment

This work was supported by grants from the National Natural Science Foundation of China (grant nos. 62377040, 62207007), Natural Science Foundation of Chongqing (grant no. CSTB2022NSCQ-MSX1256).

References

[1] S.K. Verma, J.L. Willetts, H.L. Corns, H.R. Marucci-Wellman, D.A. Lombardi, T.K. Courtney, Falls and fall-related injuries among community-dwelling adults in the United States, PLoS One 11 (3) (2016) e0150939.

[2] B. Moreland, R. Kakara, A. Henry, Trends in nonfatal falls and fall-related injuries among adults aged≥ 65 years—United States, 2012–2018, Morbidity Mortal. Wkly Rep. 69 (27) (2020) 875.

[3] M.E. Karar, H.I. Shehata, O. Reyad, A survey of IoT-based fall detection for aiding elderly care: sensors, methods, challenges and future trends, Appl. Sci. 12 (7) (2022) 3276.

[4] G. Wang, Q. Li, L. Wang, Y. Zhang, Z. Liu, CMFALL: A cascade and parallel multi-state fall detection algorithm using waist-mounted tri-axial accelerometer signals, IEEE Trans. Consum. Electron. 66 (3) (2020) 261–270.

[5] X. Yu, H. Qiu, S. Xiong, A novel hybrid deep neural network to predict pre-impact fall for older people based on wearable inertial sensors, Front. Bioeng. Biotechnol. 8 (2020) 63.

[6] W.C. Hayes, E. Myers, S. Robinovitch, A. Van Den Kroonenberg, A.C. Courtney, T. McMahon, Etiology and prevention of age-related hip fractures, Bone 18 (1) (1996) S77–S86.

[7] R. Parmar, S. Trapasiya, A comprehensive survey of various approaches on human fall detection for elderly people, Wirel. Pers. Commun. 126 (2) (2022) 1679–1703.

[8] X. Wu, Y. Zheng, C.-H. Chu, L. Cheng, J. Kim, Applying deep learning technology for automatic fall detection using mobile sensors, Biomed. Signal Process. Control 72 (2022) 103355.

[9] S. Ashry, T. Ogawa, W. Gomaa, CHARM-deep: continuous human activity recognition model based on deep neural network using IMU sensors of smartwatch, IEEE Sens. J. 20 (15) (2020) 8757–8770.

[10] D. Pan, H. Liu, D. Qu, Z. Zhang, Human falling detection algorithm based on multisensor data fusion with SVM, Mob. Inf. Syst. 2020 (2020) 1–9.

[11] P. Nandi, A. Bajaj, K. Anupama, Application of KNN for fall detection on qualcomm SoCs, in: IoT Technologies for HealthCare: 9th EAI International Conference, HealthyIoT 2022, Braga, Portugal, November 16-18, 2022, Proceedings, Springer, 2023, pp. 148–169.

[12] W.-J. Chang, L.-B. Chen, J.-P. Su, M.-C. Chen, T.-C. Yang, A fall risk prediction system based on 3D space human skeleton torso images, in: 2021 IEEE International Conference on Consumer Electronics, ICCE, IEEE, 2021, pp. 1–2.

[13] H. Li, A. Shrestha, H. Heidari, J. Le Kernec, F. Fioranelli, Bi-LSTM network for multimodal continuous human activity recognition and fall detection, IEEE Sens. J. 20 (3) (2019) 1191–1201.

[14] J. Shi, D. Chen, M. Wang, Pre-impact fall detection with CNN-based class activation mapping method, Sensors 20 (17) (2020) 4750.

[15] X. Yu, S. Park, D. Kim, E. Kim, J. Kim, W. Kim, Y. An, S. Xiong, A practical wearable fall detection system based on tiny convolutional neural networks, Biomed. Signal Process. Control 86 (2023) 105325.

[16] T.-H. Chi, K.-C. Liu, C.-Y. Hsieh, Y. Tsao, C.-T. Chan, Prefallkd: Pre-impact fall detection via CNN-ViT knowledge distillation, in: ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2023, pp. 1–5.

[17] R. Jain, V.B. Semwal, A novel feature extraction method for preimpact fall detection system using deep learning and wearable sensors, IEEE Sens. J. 22 (23) (2022) 22943–22951.

[18] J. Liu, X. Li, G. Liao, S. Wang, L. Liu, MCTN: A multi-channel temporal network for wearable fall prediction, in: Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, 2023, pp. 394–409.

[19] L. Zhao, Event prediction in the big data era: A systematic survey, ACM Comput. Surv. 54 (5) (2021) 1–37.

[20] C. Sun, S. Li, D. Cao, F.-Y. Wang, A. Khajepour, Tabular learning-based traffic event prediction for intelligent social transportation system, IEEE Trans. Comput. Soc. Syst. (2022).

[21] N. Otanasap, Pre-impact fall detection based on wearable device using dynamic threshold model, in: 2016 17th International Conference on Parallel and Distributed Computing, Applications and Technologies, PDCAT, IEEE, 2016, pp. 362–365.

[22] T.R. Mauldin, M.E. Canby, V. Metsis, A.H. Ngu, C.C. Rivera, SmartFall: A smartwatch-based fall detection system using deep learning, Sensors 18 (10) (2018) 3363.

[23] G. Vavoulas, C. Chatzaki, T. Malliotakis, M. Pediaditis, M. Tsiknakis, The mobiact dataset: Recognition of activities of daily living using smartphones, in: International Conference on Information and Communication Technologies for Ageing Well and E-Health, Vol. 2, SciTePress, 2016, pp. 143–151.

[24] A. Sucerquia, J.D. López, J.F. Vargas-Bonilla, SisFall: A fall and movement dataset, Sensors 17 (1) (2017) 198.

[25] X. Yu, J. Jang, S. Xiong, A large-scale open motion dataset (KFall) and benchmark algorithms for detecting pre-impact fall of the elderly using wearable inertial sensors, Front. Aging Neurosci. 13 (2021) 692865.

[26] V. Cotechini, A. Belli, L. Palma, M. Morettini, L. Burattini, P. Pierleoni, A dataset for the development and optimization of fall detection algorithms based on wearable sensors, Data Brief 23 (2019).

[27] S. Laal, P. Vasilyev, S. Pearson, M. Aboy, J. McNames, Feasibility of tracking human kinematics with simultaneous localization and mapping (SLAM), Sensors 22 (23) (2022) 9378.

[28] J. Li, X. Liu, Z. Wang, T. Zhang, S. Qiu, H. Zhao, X. Zhou, H. Cai, R. Ni, A. Cangelosi, Real-time hand gesture tracking for human–computer interface based on multi-sensor data fusion, IEEE Sens. J. 21 (23) (2021) 26642–26654.

[29] M. Li, S. Chen, Y. Zhao, Y. Zhang, Y. Wang, Q. Tian, Dynamic multiscale graph neural networks for 3d skeleton based human motion prediction, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 214–223.

[30] S. Shimada, V. Golyanik, W. Xu, C. Theobalt, Physcap: Physically plausible monocular 3d motion capture in real time, ACM Trans. Graph. (ToG) 39 (6) (2020) 1–16.

[31] Z. Cheng, Y. Yang, W. Wang, W. Hu, Y. Zhuang, G. Song, Time2graph: Revisiting time series modeling with dynamic shapelets, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 34, (04) 2020, pp. 3617–3624.

[32] L. Ye, E. Keogh, Time series shapelets: a new primitive for data mining, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 947–956.

[33] N. Pannurat, S. Thiemjarus, E. Nantajeewarawat, A hybrid temporal reasoning framework for fall monitoring, IEEE Sens. J. 17 (6) (2017) 1749–1759.
[34] G. Tomasch, K. Winer, Limits of absolute heading accuracy using inexpensive mems sensors, 2019.
[35] H. Jung, B. Koo, J. Kim, T. Kim, Y. Nam, Y. Kim, Enhanced algorithm for the detection of preimpact fall for wearable airbags, Sensors 20 (5) (2020) 1277.
[36] S.O. Madgwick, A.J. Harrison, R. Vaidyanathan, Estimation of IMU and MARG orientation using a gradient descent algorithm, in: 2011 IEEE International Conference on Rehabilitation Robotics, IEEE, 2011, pp. 1–7.
[37] P.L.M. Bouttefroy, A. Bouzerdoum, S.L. Phung, A. Beghdadi, On the analysis of background subtraction techniques using Gaussian mixture models, in: 2010 IEEE International Conference on Acoustics, Speech and Signal Processing, IEEE, 2010, pp. 4042–4045.
[38] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, PMLR, 2017, pp. 1263–1272.
[39] Z. Wu, M. Huang, A. Zhao, et al., Traffic prediction based on GCN-LSTM model, J. Phys.: Conf. Ser. 1972 (1) (2021) 012107.